API: ESERCITAZIONI

STRUTTURE DATI VI

GRAFI

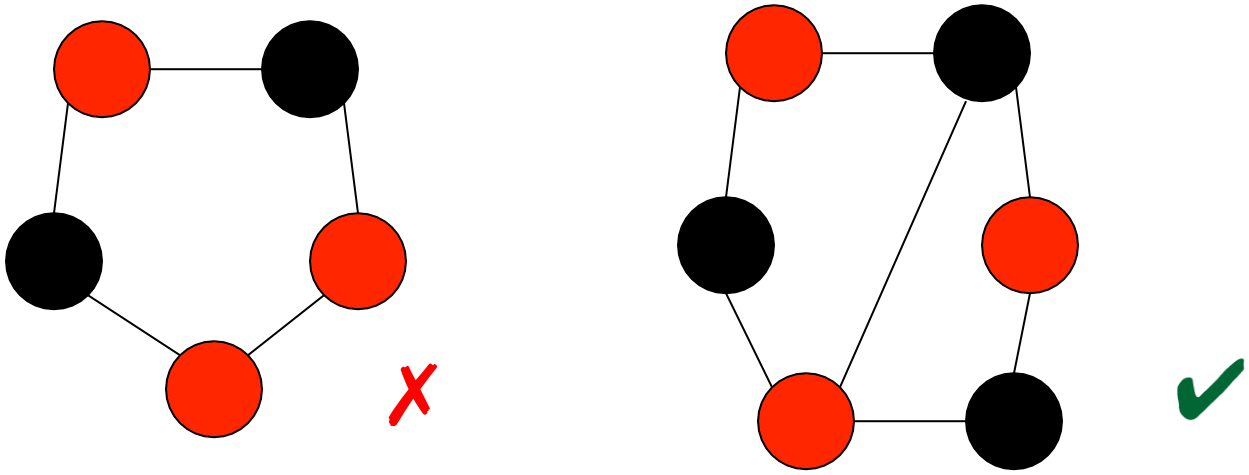
Gabriele Paveri-Fontana

Mail: gabriele.paveri@polimi.it

Es.1

Un grafo (connesso non orientato) $G = (V, E)$ è *bipartito* se V può essere partizionato in due sottoinsiemi X, Y tali che ogni arco in E collega un nodo $x \in X$ ed un nodo $y \in Y$.

Esempio:



Scrivere un algoritmo che prende in input un grafo $G = (V, E)$ e restituisce $TRUE$ se G è bipartito, $FALSE$ altrimenti.

Es.1

Scrivere un algoritmo che prende in input un grafo $G = (V, E)$ e restituisce $TRUE$ se G è bipartito, $FALSE$ altrimenti.

```
BIPARTITE(G):
 for each u in G.V
   u.color := UNDEF
 for each u in G.V
   if u.color = UNDEF
     if CHECK-VERTEX(u,RED) = FALSE
       return FALSE
 return TRUE
```

```
CHECK-VERTEX(u,color):
 u.color := color
 for each v in u.Adj
      if v.color = UNDEF
           if u.color = RED
                v.color := BLACK
           else v.color = RED
           CHECK-VERTEX(v,v.color)
      if u.color = v.color
           return FALSE
 return TRUE
```

Es.2

Si consideri il problema di stabilire se il linguaggio accettato da un NFSA sia vuoto o meno.

1) Definire una struttura dati opportuna per descrivere un NFSA.
2) Descrivere un algoritmo che prende in input un NFSA A e restituisce $TRUE$ se il linguaggio accettato da A è vuoto, $FALSE$ altrimenti.

Es.2

1) Definire una struttura dati opportuna per descrivere un NFSA A.

- Utilizziamo un grafo orientato G_A in cui ogni nodo corrisponde a uno stato ed ogni arco a una transizione. Utilizziamo una lista di adiacenza per rappresentare il grafo in memoria.
- Il grafo G_A deve avere un attributo $G_A.initial$, che è un puntatore al nodo corrispondente allo stato iniziale; inoltre ogni nodo u deve avere un attributo booleano $u.final$ che memorizza se u corrisponde ad uno stato finale o meno.
- Per ottenere una descrizione completa, dovremmo anche etichettare gli archi (in base agli input). Un arco può anche avere un'etichetta multipla nel caso in cui tra i due stati corrispondenti ai due nodi collegati dall'arco ci siano più transizioni.

Es.2

2) Descrivere un algoritmo che prende in input un NFSA A e restituisce $TRUE$ se il linguaggio accettato da A è vuoto, $FALSE$ altrimenti.

- Basta controllare se ci sia uno stato finale raggiungibile dallo stato iniziale, ovvero se tra i nodi u raggiungibili da $G_A.initial$ ce ne sia almeno uno tale che $u.final = TRUE$. In tal caso, l'algoritmo deve restituire $FALSE$, altrimenti $TRUE$.
- Dunque basta implementare un BFS con delle piccole modifiche:
  - ✓ Tutte le volte che un nodo u viene colorato di grigio, l'algoritmo deve controllare se $u.final = TRUE$; in tal caso, deve subito terminare restituendo $FALSE$.
  - ✓ Se esce dal ciclo while, l'algoritmo deve terminare restituendo $TRUE$.
- *Per esercizio*: scrivere lo pseudocodice dell'algoritmo descritto.

Es.3

Un grafo *pesato* è un grafo in cui ad ogni arco è associato un peso, ovvero un numero intero.

La *lunghezza* (il peso) di un cammino è dato dalla somma dei pesi degli archi percorsi.

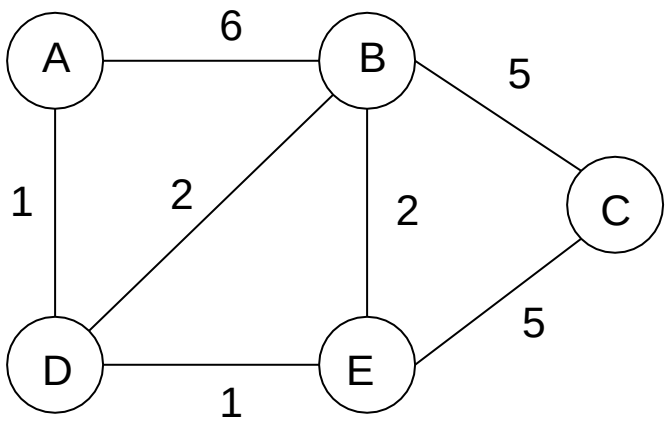
Scrivere un algoritmo che, preso in input un grafo pesato non orientato $G = (V, E)$ ed un suo nodo, detto nodo sorgente, trova i percorsi più brevi dalla sorgente ad ogni altro nodo del grafo.

Es.3

Scrivere un algoritmo che, preso in input un grafo pesato non orientato $G = (V, E)$ ed un nodo sorgente $source$, trova i percorsi più brevi dalla sorgente ad ogni altro nodo del grafo.

Algoritmo di Dijkstra

visited = [] unvisited = [B,C,D,E]



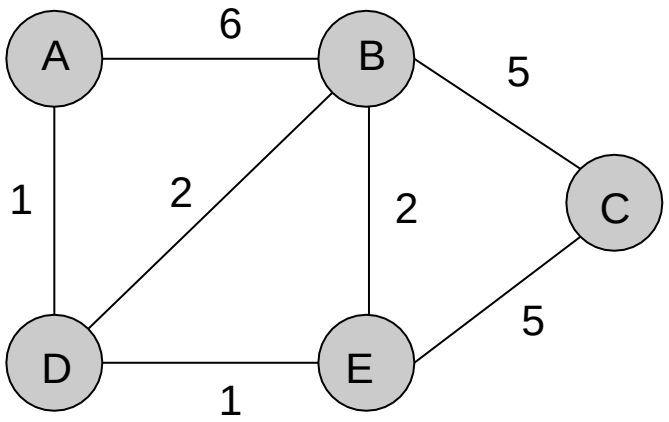
Vertex	Shortest distance	Path
A	0	A
B	∞	
C	∞	
D	∞	
E	∞	

Es.3

Scrivere un algoritmo che, preso in input un grafo pesato non orientato $G = (V, E)$ ed un nodo sorgente $source$, trova i percorsi più brevi dalla sorgente ad ogni altro nodo del grafo.

Algoritmo di Dijkstra

visited = [D, E, B, C] unvisited = []



Vertex	Shortest distance	Path
A	0	A
B	3	A->D
C	7	A->D->E
D	1	A
E	2	A->D

Es.3

Scrivere un algoritmo che, preso in input un grafo pesato non orientato $G = (V, E)$ ed un nodo sorgente $source$, trova i percorsi più brevi dalla sorgente ad ogni altro nodo del grafo.

Algoritmo di Dijkstra

```
DIJKSTRA(G,source):
 dist[source] := 0
 S := ∅
 for each u in G.V
       if (u ≠ source) dist[u] := ∞
       add u to S
 while S ≠ ∅
       u := element of S with min dist[u]
       remove u from S
       for each v in u.Adj ∩ S
             alt := dist[u] + length(u,v)
             if alt < dist[v]
                   dist[v] := alt
 return dist
```

Es.4

Dato un grafo connesso non orientato G, uno *spanning tree* di G è un sottografo che è un albero e collega tutti i nodi di G.

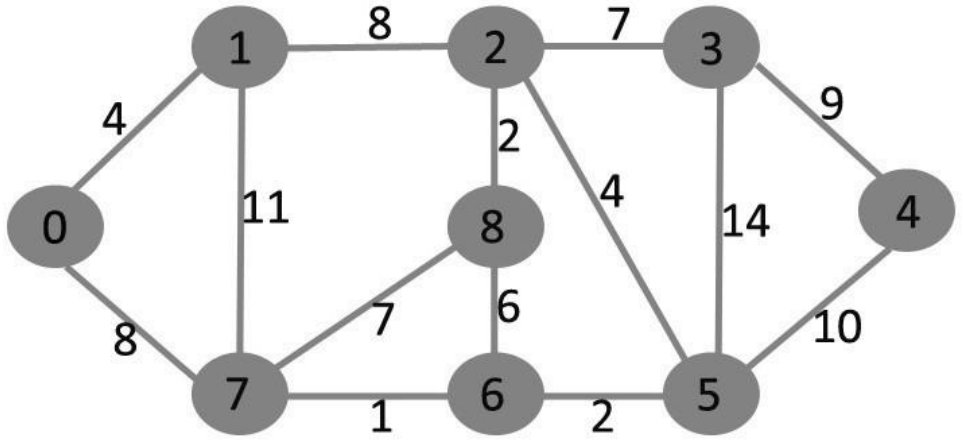
Uno stesso grafo G può avere diversi spanning trees.

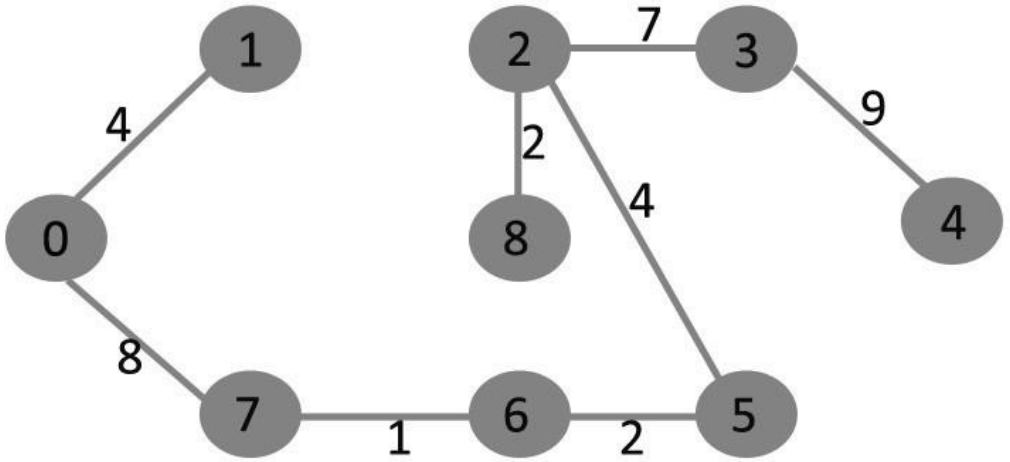
Dato un grafo pesato connesso non orientato G, uno spanning tree minimo (MST) di G è uno spanning tree il cui peso è minore o uguale al peso di tutti gli altri spanning trees di G (dove il peso di uno spanning tree è la somma dei pesi di tutti i suoi archi).

Scrivere un algoritmo che, preso in input un grafo pesato connesso non orientato $G = (V, E)$, restituisce un suo MST.

Es.4

Esempio:





Es.4

Scrivere un algoritmo che, preso in input un grafo pesato connesso non orientato $G = (V, E)$, restituisce un suo MST.

Algoritmo di Kruskal

- Crea una foresta (un insieme di alberi), in cui ogni nodo di G è un albero a sé stante.
- Crea un insieme S inizialmente vuoto.
- Ordina gli elementi di E in ordine crescente.
- Per ogni arco (u, v) di E, se (u, v) connette due alberi diversi della foresta, aggiunge (u, v) ad S ed alla foresta (connettendo così due alberi).
- Quando l'algoritmo termina, l'insieme S contiene gli archi di un MST di G.

Es.4

Scrivere un algoritmo che, preso in input un grafo pesato connesso non orientato $G = (V, E)$, restituisce un suo MST.

Algoritmo di Kruskal

```
KRUSKAL(G):
 for each u in G.V
      MAKESET(u)
 S := ∅
 sort G.E by weight in increasing order
 for each (u,v) in G.E
      x := FIND(u), y := FIND(v)
      if x ≠ y
           add (u,v) to S
           UNION(x,y)
```

```
MAKESET(u):
 u.p := u

FIND(u):
 while u ≠ u.p
      u := u.p
 return u

UNION(x,y):
 x.p := y
```

Es.5

Scrivere un algoritmo che generi randomicamente un grafo $G = (V, E)$, con $|V| = n$, dove:

- Non ci sono vincoli sul numero totale di archi.
- Due nodi possono essere connessi da al più un arco.
- Due nodi vengono connessi da un arco secondo una certa probabilità p.

Es.5

Scrivere un algoritmo che generi randomicamente un grafo $G = (V, E)$, con $|V| = n$, dove:

Modello Erdös-Rényi

```
ERDOS-RENYI(n,p):
 V := ∅
 E := ∅
 for i in range (1..n)
    create node v
    V := V union v
 for v in V
    for w in V
       if v ≠ w
          q := random(0,1)
          create edge <v,w>
          if q <= p and not (<v,w> in E)
             E := E union <v,w>
 return (V,E)
```

Es.5

Osservazioni e definizioni

- Sia dato un grafo $G = (V, E)$, non orientato e non pesato. Il *grado* di un nodo $v \in V$, indicato con $\deg(v)$, è definito come il numero totale di archi incidenti in v, mentre il *grado* del grafo G, indicato come $\deg(G)$, è definito come la somma del grado di tutti i suoi nodi, ossia $\deg(G) = \sum_{v \in V} \deg(v)$.
- Nei sistemi reali capita che pochi nodi (chiamati *hub*, nodi con grado elevato) siano connessi ad una grande quantità di altri nodi, mentre tutti gli altri nodi siano scarsamente connessi (grado basso). Questi sistemi vengono chiamati *scale-free*.
- Il **modello Erdös-Rényi** per la generazione di grafi randomici non è molto rappresentativo dei sistemi scale-free. Infatti, nei grafi che esso genera il grado dei nodi è quasi del tutto uniforme.
- Un modello che è noto per catturare le caratteristiche dei sistemi scale-free è il **modello Barabási-Albert**. Approfondite!